Assignment 5: DBSCAN From Scratch — QuickBite Profit Optimization

4 graded steps (20/20/20/40) • Submit ONE PDF (include code + plots + printed results)

Scenario (business case)

QuickBite is a food-delivery company. Each order has a drop-off location (x, y).

Business problem:
• Long travel distance increases driver cost (fuel/time).
• Late deliveries create refunds/discounts.

Goal: Increase profit by opening exactly TWO new micro-kitchens near high-demand hotspots. DBSCAN is used because hotspots can have irregular shapes (for example, a corridor along a main road) and DBSCAN can also detect noise (rare/suspicious orders).

Rules (strict)

- You MUST implement DBSCAN from scratch (NumPy only).
- Forbidden: sklearn.cluster.DBSCAN, any completed DBSCAN library, NearestNeighbors, or any clustering library that outputs final labels.
- Allowed: numpy, matplotlib (optional: pandas only for tables).
- Your PDF must include all code + plots + printed outputs. Optional .py/.ipynb may be attached but grading uses the PDF.

Submission (ONE PDF)

Submit ONE PDF. Your PDF must have 4 clearly separated parts: Step 1, Step 2, Step 3, Step 4.
I will score each step separately using the rubric.

Step 1 (20 pts) — Generate dataset + standardize + plot before clustering

Run this code to generate order locations, standardize with NumPy, and plot in gray.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(7)

# Dense order hotspots (neighborhoods)
hubs = [
    ((200, 820), 30, 280),  # Uptown
    ((760, 760), 25, 240),  # Business district
    ((240, 260), 40, 320),  # Residential
    ((820, 260), 35, 260),  # University
]

points = []
for (cx, cy), sigma, n in hubs:
    points.append(np.random.normal(loc=(cx, cy), scale=sigma, size=(n,
2)))

# Main road / delivery corridor (irregular shape)
n_corridor = 260
t = np.random.rand(n_corridor, 1)
corridor = np.hstack([400 + 220*t, 120 + 760*t])
corridor += np.random.normal(scale=18, size=corridor.shape)
points.append(corridor)

# Outliers / low-demand / suspicious orders
n_noise = 180
points.append(np.random.uniform(low=(0, 0), high=(1000, 1000),
size=(n_noise, 2)))

X = np.vstack(points)

# Scale with NumPy only: z = (x - mean) / std
mu = X.mean(axis=0)
std = X.std(axis=0)
X_scaled = (X - mu) / std

plt.figure(figsize=(6.5, 6.5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c="gray", s=12)
plt.title("Step 1: Order Locations Before Clustering (Scaled)")
plt.xlabel("x (scaled)")
plt.ylabel("y (scaled)")
plt.grid(True)
plt.show()
```

What to submit (Step 1):

- Step 1 plot (gray points).
- Step 1 code (dataset + scaling).

Step 2 (20 pts) — DBSCAN from scratch (fixed eps/min) + core/border/noise counts

Implement DBSCAN from scratch and run it with:

- eps = 0.18
- min_samples = 10

Required printed results:

- #clusters
- #core points
- #border points
- #noise points
- %noise

Definitions (use exactly these words):

- Core point: a non-noise point with at least min_samples points (including itself) in its eps-neighborhood.
- Border point: a non-core point (not core) that still belongs to a cluster.
- Noise point: a point that does not belong to any cluster (label = -1).

```
import numpy as np

UNASSIGNED = -99
NOISE = -1

def region_query(X, point_idx, eps):
    d = np.linalg.norm(X - X[point_idx], axis=1)
    return np.where(d <= eps)[0]

def dbscan_scratch(X, eps, min_samples):
    n = len(X)
    labels = np.full(n, UNASSIGNED, dtype=int)
    visited = np.zeros(n, dtype=bool)
    cluster_id = 0

    for i in range(n):
        if visited[i]:
            continue
        visited[i] = True

        neighbors = region_query(X, i, eps)
        if len(neighbors) < min_samples:
            labels[i] = NOISE
            continue

        labels[i] = cluster_id
        queue = list(neighbors)

        while queue:
            j = queue.pop(0)

            # IMPORTANT: a point labeled NOISE can later become a cluster
point (border) if reached from a core point
```

```
            if labels[j] == NOISE:
                labels[j] = cluster_id

            if not visited[j]:
                visited[j] = True
                j_neighbors = region_query(X, j, eps)
                if len(j_neighbors) >= min_samples:
                    for k in j_neighbors:
                        if k not in queue:
                            queue.append(k)

            if labels[j] in (UNASSIGNED, NOISE):
                labels[j] = cluster_id

        cluster_id += 1

    labels[labels == UNASSIGNED] = NOISE
    return labels


import numpy as np
import matplotlib.pyplot as plt

eps = 0.18
min_samples = 10

labels = dbscan_scratch(X_scaled, eps=eps, min_samples=min_samples)

unique_labels = sorted(set(labels))
n_clusters = len([lab for lab in unique_labels if lab != -1])
n_noise = int(np.sum(labels == -1))
noise_pct = 100 * n_noise / len(labels)

# Core vs Border counts (border = non-core point)
core = 0
border = 0
for i in range(len(X_scaled)):
    if labels[i] == -1:
        continue
    neighbors = region_query(X_scaled, i, eps)
    if len(neighbors) >= min_samples:
        core += 1
    else:
        border += 1

print("Clusters:", n_clusters)
print("Core points:", core)
print("Border points:", border)
print("Noise points:", n_noise)
print("Noise %:", round(noise_pct, 2))

plt.figure(figsize=(6.5, 6.5))
for lab in unique_labels:
    mask = (labels == lab)
```

```
    if lab == -1:
        plt.scatter(X_scaled[mask, 0], X_scaled[mask, 1], marker='x',
s=28, label='Noise')
    else:
        plt.scatter(X_scaled[mask, 0], X_scaled[mask, 1], s=12,
label=f'Cluster {lab}')

plt.title(f"Step 2: DBSCAN (eps={eps}, min_samples={min_samples})")
plt.xlabel("x (scaled)")
plt.ylabel("y (scaled)")
plt.legend()
plt.grid(True)
plt.show()
```

What to submit (Step 2):

- DBSCAN-from-scratch code.
- Step 2 plot (clusters + noise).
- Printed outputs: #clusters, #core, #border, #noise, %noise.
- Explanation (120–170 words): explain how core/border/noise relates to hotspot strength and profit.

Step 3 (20 pts) — Parameter study (9 cases)

Test all 9 combinations:
eps ∈ {0.14, 0.18, 0.22}
min_samples ∈ {10, 15, 20}

For each case, show a plot and report #clusters and #noise. Then make one comparison table (9 rows).

```
import numpy as np
import matplotlib.pyplot as plt

eps_list = [0.14, 0.18, 0.22]
min_list = [10, 15, 20]

fig, axes = plt.subplots(3, 3, figsize=(10.5, 10.0))
records = []

for i, min_samples in enumerate(min_list):
    for j, eps in enumerate(eps_list):
        labels = dbscan_scratch(X_scaled, eps=eps,
min_samples=min_samples)
        unique_labels = sorted(set(labels))

        n_clusters = len([lab for lab in unique_labels if lab != -1])
        n_noise = int(np.sum(labels == -1))
        noise_pct = 100 * n_noise / len(labels)

        records.append([eps, min_samples, n_clusters, n_noise, noise_pct])

        ax = axes[i, j]
        for lab in unique_labels:
            mask = (labels == lab)
            if lab == -1:
                ax.scatter(X_scaled[mask, 0], X_scaled[mask, 1],
marker='x', s=10)
            else:
                ax.scatter(X_scaled[mask, 0], X_scaled[mask, 1], s=7)

        ax.set_title(f"eps={eps}, min={min_samples}\nC={n_clusters},
N={n_noise} ({noise_pct:.1f}%)", fontsize=10)
        ax.grid(True)
        ax.set_xlabel("x")
        ax.set_ylabel("y")

plt.tight_layout()
plt.show()

print("eps, min_samples, #clusters, #noise, %noise")
for r in records:
    print(r[0], r[1], r[2], r[3], round(r[4], 2))
```

What to submit (Step 3):

- Plots for all 9 cases (9 plots or one clear 3×3 grid plot).
- Short note per case (1–2 lines) with #clusters and #noise.
- Comparison table (9 rows): eps, min_samples, #clusters, #noise, %noise.

Step 4 (40 pts) — Apply the profit rule to pick ONE best (eps,min) + recommend 2 micro-kitchens

Profit rule (must satisfy ALL):
A) #clusters must be exactly 5 (4 neighborhoods + 1 main-road corridor)
B) noise% must be ≤ 9.0%
C) min_samples must be ≤ 15

From Step 3, exactly ONE (eps, min_samples) will satisfy A–C. That is the unique best answer.

```
import numpy as np
import pandas as pd

# Replace with the UNIQUE pair you found from Step 3
eps = 0.14
min_samples = 10

labels = dbscan_scratch(X_scaled, eps=eps, min_samples=min_samples)

# Cluster summary using ORIGINAL coordinates X
clusters = sorted([lab for lab in set(labels) if lab != -1])
rows = []
for lab in clusters:
    pts = X[labels == lab]
    rows.append([lab, len(pts), pts[:,0].mean(), pts[:,1].mean()])

summary = pd.DataFrame(rows, columns=["cluster_id", "points",
"centroid_x", "centroid_y"])
summary = summary.sort_values("points", ascending=False)
print(summary)

# Choose best TWO centroids by trying all pairs (minimize total distance)
served = X[labels != -1]
centroids = [(int(r.cluster_id), np.array([r.centroid_x, r.centroid_y]))
for _, r in summary.iterrows()]

best = None
for i in range(len(centroids)):
    for j in range(i+1, len(centroids)):
        cid1, c1 = centroids[i]
        cid2, c2 = centroids[j]
        total_m = np.minimum(np.linalg.norm(served - c1, axis=1),
np.linalg.norm(served - c2, axis=1)).sum()
        if best is None or total_m < best[0]:
            best = (total_m, cid1, cid2, c1, c2)

total_m, cid1, cid2, c1, c2 = best
print("Best pair cluster IDs:", cid1, cid2)
print("Kitchen 1 centroid:", c1)
print("Kitchen 2 centroid:", c2)

# Profit estimate vs old single kitchen at (500,500)
old = np.array([500.0, 500.0])
```

```
baseline_m = np.linalg.norm(served - old, axis=1).sum()
improved_m = total_m

reduction_km = (baseline_m - improved_m) / 1000.0
daily_saving_thb = reduction_km * 30.0

print("Distance saving (km):", round(reduction_km, 2))
print("Estimated daily saving (THB):", round(daily_saving_thb, 2))
```

What to submit (Step 4):

- Show which ONE (eps, min_samples) satisfies A–C (use your Step 3 table).
- Final DBSCAN plot using that parameter set.
- Printed outputs: #clusters, #noise, %noise.
- Cluster centroid table (original coordinates).
- Best 2 micro-kitchen centroids + distance saving + estimated saving.
- Explanation (250–350 words): why this increases profit + how to handle noise orders.

Grading rubric (20 / 20 / 20 / 40)

Step	What is graded (must be in your PDF)	Points
1	Dataset + scaling + plot before clustering	20
2	DBSCAN scratch + plot + #clusters/#core/#border/#noise + explanation	20
3	9-case plots + notes + comparison table	20
4	Unique parameter selection + centroids + profit estimate + explanation	40